Meeting Minutes

Meeting Details

- **Meeting title:** TestLab UX and MVP Brainstorm
- **Date:** 2026-05-10
- **Time:** 60 minutes (strategy session)
- **Location:** AI Team Coordination Session
- **Facilitator:** TestLab AI Master
- **Minute taker:** TestLab AI Master

Attendees

- Chief Architect (Customer-facing decision owner)
- testlab-ai-master (Facilitator)
- testlab-architect (Architecture)
- testlab-ide-master (Frontend UX and block system)
- testlab-master (Backend and SDK alignment)
- testlab-test-master (Execution reliability and quality)

Agenda

1. Review customer pain points in current UX and templates.
2. Compare target behavior with the test-orchestrator reference.
3. Brainstorm predefined capability model for MVP.
4. Align on risks, scope, and immediate next actions.

Customer Pain Points Captured

- Creating test cases is too complicated for users.
- Template structure is confusing and unclear about required inputs.
- Users need predefined capabilities instead of composing low-level building blocks from scratch.
- YAML output does not align clearly with Python SDK execution reality.
- Backend step organization is not useful enough for running practical examples.
- MVP expectation: match 1:1 core functionality of test-orchestrator, but in a reusable generic structure.

Discussion Summary

1) UX and Template Complexity

- Current flow exposes technical primitives too early.
- Users think in workflows (for example, negotiate contract and transfer), but UI presents tool-level blocks.
- Required versus optional inputs are not obvious, increasing cognitive load.

2) Predefined Capabilities Direction

- Team aligned on introducing predefined workflow capabilities as the MVP center.
- Capability examples discussed:
  - Catalog query and negotiation workflow
  - Data transfer workflow
  - Digital twin lookup workflow
- Capabilities should expose minimal required inputs and safe defaults.

3) YAML and SDK Alignment

- Current abstraction layer can hide SDK behavior and create mismatch.
- Team agreed capability definitions must map clearly to executable backend behavior.
- Output YAML must remain auditable and understandable by users.

4) Backend Execution Model

- MVP should prioritize deterministic sequential execution for reliability.
- Clear failure context is mandatory (which step failed, expected result, actual result).
- Cleanup strategy should be defined for partial failures.

5) Test and Reliability Expectations

- Each capability should have dedicated integration tests.
- Include happy path and failure-path validation.
- Avoid parallel step execution in MVP to reduce complexity.

Decisions

1. **MVP strategy:** capability-first UX approach.
2. **Execution approach:** sequential, deterministic workflows in MVP.
3. **Quality baseline:** built-in validation and clearer failure reporting per capability.
4. **Scope discipline:** start with a small, high-confidence set of capabilities.

Open Decisions (Chief Architect)

1. Final list of first 3 MVP capabilities.
2. Capability authoring format for MVP (YAML-first proposed).
3. Whether MVP capabilities are curated-only or user-extensible.
4. Timeline and scope cap for first release.

Action Items

- **Chief Architect:** confirm MVP scope and first 3 capabilities.
- **testlab-architect:** propose capability contract (inputs, outputs, defaults, assertions).
- **testlab-ide-master:** propose UX surface for capability selection and simplified input forms.
- **testlab-master:** draft backend mapping from capability definitions to executable steps.
- **testlab-test-master:** define integration test matrix for each capability (happy path + failure path + cleanup).
- **testlab-ai-master:** coordinate work packages after scope confirmation.

Risks and Mitigations

- **Risk:** Over-engineering capability abstraction too early.
  - **Mitigation:** Keep MVP to 3 capabilities and validate with real customer scenarios.
- **Risk:** YAML and SDK drift over time.
  - **Mitigation:** define explicit capability-to-execution contract and version checks.
- **Risk:** UX simplification blocks advanced users.
  - **Mitigation:** capability-first default with later advanced mode.

Success Criteria for MVP

- Users can create and run common scenarios without assembling everything manually.
- Required inputs are clear and minimal.
- Generated YAML is readable and consistent with execution behavior.
- Failures are diagnosable with step-level error context.
- Capability library can be expanded without redesign.

Next Meeting

- **Proposed date:** within 3 business days after scope confirmation.
- **Purpose:** approve capability contract and implementation work packages.